

Fondation
UCAD



UNIVERSITE
CHEIKH ANTA DIOP
DE DAKAR (UCAD)

ECOLE SUPERIEURE POLYTECHNIQUE



Département Génie Informatique

Systèmes Réseaux et Télécommunications

Licence 3 en Systèmes Réseaux et Télécommunications

SDN SDR NFV

ETUDIANTS :

Professeur : **Dr Sadio**

El Hadji Mamadou **NDIAYE**

RAPPORT DE DEPLOIEMENT ET VALIDATION

1. Introduction et Contexte

Ce rapport presente le deploiement complet de CleanPipe, une passerelle de securite cloudbative deployee sur un cluster Kubernetes leger (K3s) dans un environnement virtualise Vagrant/Ubuntu 20.04. Le projet met en oeuvre une architecture NFV (Network Functions Virtualization) conforme au standard ETSI, avec un Service Function Chaining (SFC) compose de trois fonctions reseau conteneurisees (CNF).

La chaine de service implementee est la suivante :

- CLIENT (Alpine/curl) — origine du trafic utilisateur via net1 (10.10.1.1/24)
- FRR (Firewall/Router) — filtrage ACL SSH et routage SFC via net1/net2
- SURICATA (IDS/DPI) — inspection profonde des paquets et detection d'intrusion via net1/net2
- NGINX (Web Server) — service web destination finale via net1 (10.10.3.4/24)

1.1 Diagramme d'Architecture

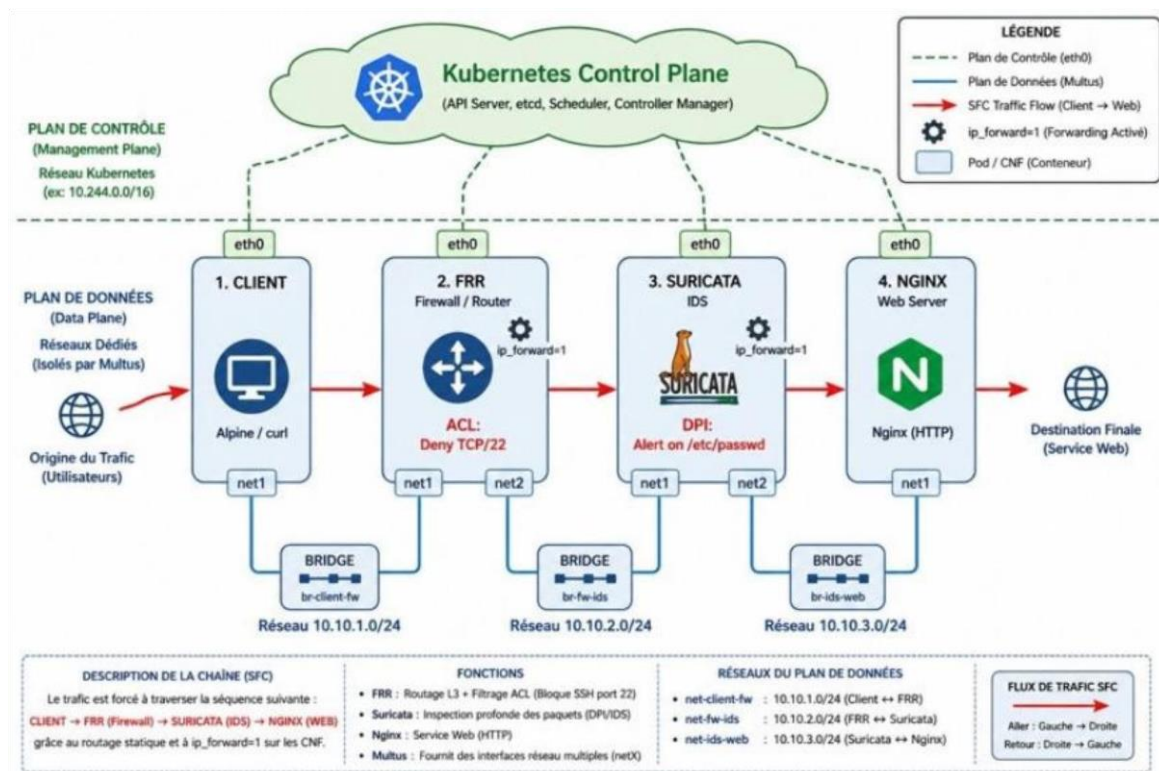


Figure 1 — Architecture NFV/SFC CleanPipe avec Multus CNI et bridges Linux

2. Manifests Kubernetes (.yaml)

Cette section présente les captures complètes des manifests de l'ensemble des Pods et Services Kubernetes de l'architecture CleanPipe en cours d'exécution, conformément aux exigences du projet.

2.1 Manifests des Pods — `kubectl get pod -o yaml`

Les pods sont déployés avec les annotations Multus CNI pour attacher les interfaces réseau

secondaires (net1, net2) sur les bridges Linux dédiés à la chaîne SFC.

```
vagrant@comnetsemu:~/cleanpipe$ kubectl get pod -o yaml
apiVersion: v1
items:
- apiVersion: v1
  kind: Pod
  metadata:
    annotations:
      k8s.v1.cni.cncf.io/networks: |
        [{
          "name": "net-client-fw",
          "interface": "net1",
          "ips": ["10.10.1.1/24"]
        }]
    kubectrl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{"k8s.v1.cni.cncf.io/networks":[{"name":"net-client-fw","interface":"net1","ips":["10.10.1.1/24"]}]},"labels":{"app":"client","name":"client"},"namespace":"default"},"spec":{"containers":[{"args":["# Installer curl\napk add --no-cache curl\n\n# Attendre les interfaces Multus\nsleep 5\n\n# Route SFC : tout trafic vers NGINX passe par FRR (net1)\nip route add 10.10.3.0/24 via 10.10.1.2 dev net1 2\u003e/dev/null || true\nip route add 10.10.2.0/24 via 10.10.1.2 dev net1 2\u003e/dev/null || true\n\nsleep infinity\n"],"command":["/bin/sh","-c"],"image":"alpine:latest","name":"client","securityContext":{"privileged":true}}],"restartPolicy":"Always"}}
    creationTimestamp: "2026-05-14T00:41:19Z"
    generation: 1
    labels:
      app: client
      name: client
      namespace: default
      resourceVersion: "109880"
```

```
  linux:
    gid: 0
    supplementalGroups:
      - 0
    uid: 0
  volumeMounts:
  - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
    name: kube-api-access-662xt
    readOnly: true
    recursiveReadOnly: Disabled
  hostIP: 192.168.40.133
  hostIPs:
  - ip: 192.168.40.133
  observedGeneration: 1
  phase: Running
  podIP: 10.42.0.102
  podIPs:
  - ip: 10.42.0.102
  qosClass: BestEffort
  startTime: "2026-05-14T00:41:19Z"
kind: List
metadata:
  resourceVersion: ""
```

✚ Pour le Client : *kubectl get pod client -o yaml*

```
vagrant@comnetsemu:~/cleanpipe$ kubectl get pod client -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    k8s.v1.cni.cncf.io/networks: |
      [{
        "name": "net-client-fw",
        "interface": "net1",
        "ips": ["10.10.1.1/24"]
      }]
    kubectrl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{"k8s.v1.cni.cncf.io/networks":[{"name":"net-client-fw","interface":"net1","ips":["10.10.1.1/24"]}]},"labels":{"app":"client","name":"client"},"namespace":"default"},"spec":{"containers":[{"args":["# Installer curl\napk add --no-cache curl\n\n# Attendre les interfaces Multus\nsleep 5\n\n# Route SFC : tout trafic vers NGINX passe par FRR (net1)\nip route add 10.10.3.0/24 via 10.10.1.2 dev net1 2\u003e/dev/null || true\nip route add 10.10.2.0/24 via 10.10.1.2 dev net1 2\u003e/dev/null || true\n\nsleep infinity\n"],"command":["/bin/sh","-c"],"image":"alpine:latest","name":"client","securityContext":{"privileged":true}}],"restartPolicy":"Always"}}
    creationTimestamp: "2026-05-14T00:41:19Z"
    generation: 1
    labels:
      app: client
```

Au démarrage, son script d'initialisation injecte des **routes statiques** dans son espace de nommage réseau pour forcer tout le trafic à destination de l'IDS et du serveur Web à transiter par la passerelle de sécurité FRR (10.10.1.2), initiant ainsi le chaînage de services (SFC).

```
vagrant@comnetsemu:~/cleanpipe$ kubectl get pod frr -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    k8s.v1.cni.cncf.io/networks: |
      [{"name": "net-client-fw",
        "interface": "net1",
        "ips": ["10.10.1.2/24"]}
      ],{
        "name": "net-fw-ids",
        "interface": "net2",
        "ips": ["10.10.2.2/24"]}
      ]
    kubectrl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{"k8s.v1.cni.cncf.io/networks":"[{\"name\": \"net-cl
        ient-fw\",\\n \"interface\": \"net1\",\\n \"ips\": [\"10.10.1.2/24\"]\\n}],{\\n \"name\": \"net-fw-ids\",\\n \"interface\"
        : \"net2\",\\n \"ips\": [\"10.10.2.2/24\"]\\n}]\\n\"},\"labels\":{\"app\":\"frr\"},\"name\":\"frr\",\"namespace\":\"default\"},\"spec\":{\"c
        ontainers\":[{\"args\":[\"sysctl -w net.ipv4.ip_forward=1\\n\\n# ACL : bloquer SSH\\niptables -A FORWARD -p tcp --dport 22 -j D
        ROP\\niptables -A INPUT -p tcp --dport 22 -j DROP\\n\\n# Attendre les interfaces Multus\\nsleep 5\\n\\n# Route SFC : trafic ve
        rs NGINX (10.10.3.0/24) via Suricata\\nip route add 10.10.3.0/24 via 10.10.2.3 dev net2 2\\u003e/dev/null || true\\n\\n# Route
```

Son script d'initialisation active le forwarding IPv4 au niveau du noyau, applique des règles iptables pour bloquer le flux SSH (TCP/22), et configure les routes statiques nécessaires pour acheminer le trafic de manière forcée vers le prochain saut de la chaîne SFC.

```
vagrant@comnetsemu:~/cleanpipe$ kubectl get pod suricata -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    k8s.v1.cni.cncf.io/networks: |
      [{
        "name": "net-fw-ids",
        "interface": "net1",
        "ips": ["10.10.2.3/24"]
      }, {
        "name": "net-ids-web",
        "interface": "net2",
        "ips": ["10.10.3.3/24"]
      }]
  kubernet.es.io/last-applied-configuration: |
    {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{"k8s.v1.cni.cncf.io/networks":[{"\n  \\"name\\": \\"net-
w-ids\\",\n  \\"interface\\": \\"net1\\",\n  \\"ips\\": [\\"10.10.2.3/24\\"]\n}],{"\n  \\"name\\": \\"net-ids-web\\",\n  \\"interface\\":
\\"net2\\",\n  \\"ips\\": [\\"10.10.3.3/24\\"]\n}}\n},"labels":{"app":"suricata"},"name":"suricata","namespace":"default"
,"spec":{"containers":[{"args":["sysctl -w net.ipv4.ip_forward=1\n\n# Règle DPI\nmkdir -p /etc/suricata/rules\ncat \u003e
/etc/suricata/rules/cleanpipe.rules \u003c\u003c 'RULES'\nalert tcp any any -\u003e any 80 (msg:\\"CLEANPIPE /etc/pa
startime: \"2026-05-14T00:41:10Z\""}]}
```

Ce manifeste montre que le pod **suricata** intègre deux interfaces réseaux secondaires fournies par Multus CNI (net1 en 10.10.2.3/24 et net2 en 10.10.3.3/24) pour se positionner à la suite du pare-feu et juste avant le serveur Web.

Son script d'initialisation configure une règle d'inspection profonde (**DPI**) ciblant l'accès au fichier sensible /etc/passwd sur le port HTTP (80), puis active le mode d'écoute passive sur l'interface d'entrée net1 tout en assurant le routage des paquets légitimes vers le serveur final Nginx.

✦ **Pour le Serveur Nginx:** *kubectl get pod nginx -o yaml*

```
vagrant@comnetsemu:~/cleanpipe$ kubectl get pod nginx -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    k8s.v1.cni.cncf.io/networks: |
      [{
        "name": "net-ids-web",
        "interface": "net1",
        "ips": ["10.10.3.4/24"]
      }]
  kubect1.kubernetes.io/last-applied-configuration: |
    {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{"k8s.v1.cni.cncf.io/networks":[{"name":"net-ids-web","interface":"net1","ips":["10.10.3.4/24"]}]},"labels":{"app":"nginx"},"name":"nginx","namespace":"default"},"spec":{"containers":[{"args":["# Activer ip_forward\nsysctl -w net.ipv4.ip_forward=1\n# Route retour vers Suricata via net1\nsleep 5\nip route add 10.10.1.0/24 via 10.10.3.3 dev net1 2\u003e/dev/null || true\nip route add 10.10.2.0/24 via 10.10.3.3 dev net1 2\u003e/dev/null || true\n\n# Démarrer nginx en foreground\nnginx -g 'daemon off;'\n"],"command":["/bin/bash","-c"],"image":"nginx:latest","name":"nginx","securityContext":{"privileged":true}}]}
  startTime: "2026-05-14T00:41:19Z"
```

Ce manifeste confirme que le pod final **nginx** est équipé d'une interface réseau secondaire Multus CNI nommée net1 avec l'adresse IP 10.10.3.4/24 pour s'interconnecter directement avec l'IDS Suricata.

Son script d'initialisation active le transfert de paquets IPv4 au niveau du noyau et configure des routes statiques de retour pour forcer les réponses HTTP à destination du client (10.10.1.0/24) à repasser par l'interface de Suricata (10.10.3.3).

Une fois cette configuration réseau appliquée, le serveur Web Nginx est démarré au premier plan afin de traiter les requêtes légitimes ayant traversé l'intégralité de la chaîne de sécurité.

L'état final phase: Running atteste du déploiement réussi et stable de ce point de terminaison de l'architecture SFC.

3. Listing des Ressources Kubernetes

3.1 Pods en cours d'exécution

Les quatre pods de l'architecture CleanPipe sont tous en état Running sur le noeud comnetsemu du cluster K3s :

Pod	Role SFC	Interface	Adresse IP	Reseau Bridge
-----	----------	-----------	------------	---------------

client	Origine du trafic	eth0 + net1	10.10.1.1/24	br-client-fw
frr	Firewall / ACL SSH	net1 / net2	10.10.1.2 / 10.10.2.2	br-client-fw / br-fw-ids
suricata	IDS / DPI	net1 / net2	10.10.2.3 / 10.10.3.3	br-fw-ids / br-ids-web
nginx	Serveur web destination	eth0 + net1	10.10.3.4/24	br-ids-web

3.2 Services Kubernetes

Les services suivants sont crees pour exposer les CNF de la chaine SFC. Le service nginxnodeport (type NodePort) permet l'accès au service web depuis l'exterieur du cluster sur le port 30080 :

Nom du Service	Type	ClusterIP	Port	Selector
svc-client	ClusterIP (headless)	None	80	app: client
svc-frr	ClusterIP (headless)	None	80	app: frr
svc-suricata	ClusterIP (headless)	None	80	app: suricata
svc-nginx	ClusterIP (headless)	None	80	app: nginx
nginx-nodeport	NodePort	10.43.190.94	30080	app: nginx

→ **Les Services de notre Architecture :**

```
vagrant@comnetsemu:~/cleanpipe$ kubectl get svc svc-client -o yaml
apiVersion: v1
kind: Service
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Service","metadata":{"annotations":{},"name":"svc-client","namespace":"default"},"spec":{"clusterIP":"None","ports":[{"port":80}],"selector":{"app":"client"}}}
  creationTimestamp: "2026-05-14T19:08:58Z"
  name: svc-client
  namespace: default
  resourceVersion: "97060"
  uid: 4f93e1d3-5a2e-4de0-a714-264b043c7cbb
spec:
  clusterIP: None
  clusterIPs:
  - None
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: client

vagrant@comnetsemu:~/cleanpipe$ kubectl get svc svc-frr -o yaml
apiVersion: v1
kind: Service
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Service","metadata":{"annotations":{},"name":"svc-frr","namespace":"default"},"spec":{"clusterIP":"None","ports":[{"port":80}],"selector":{"app":"frr"}}}
  creationTimestamp: "2026-05-14T19:08:58Z"
  name: svc-frr
  namespace: default
  resourceVersion: "97061"
  uid: bc4c07bb-98c2-4bf5-a7da-25ee0355dfa4
spec:
  clusterIP: None
  clusterIPs:
  - None
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: frr

vagrant@comnetsemu:~/cleanpipe$ kubectl get svc svc-suricata -o yaml
apiVersion: v1
kind: Service
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Service","metadata":{"annotations":{},"name":"svc-suricata","namespace":"default"},"spec":{"clusterIP":"None","ports":[{"port":80}],"selector":{"app":"suricata"}}}
  creationTimestamp: "2026-05-14T19:08:58Z"
  name: svc-suricata
  namespace: default
  resourceVersion: "97062"
  uid: 238d74ab-a12b-4963-afc3-f964138894c1
spec:
  clusterIP: None
  clusterIPs:
  - None
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: suricata

vagrant@comnetsemu:~/cleanpipe$ kubectl get svc svc-suricata -o yaml
apiVersion: v1
kind: Service
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Service","metadata":{"annotations":{},"name":"svc-suricata","namespace":"default"},"spec":{"clusterIP":"None","ports":[{"port":80}],"selector":{"app":"suricata"}}}
  creationTimestamp: "2026-05-14T19:08:58Z"
  name: svc-suricata
  namespace: default
  resourceVersion: "97062"
  uid: 238d74ab-a12b-4963-afc3-f964138894c1
spec:
  clusterIP: None
  clusterIPs:
  - None
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: suricata
```

Ces manifestes montrent la définition des quatre services Kubernetes (svc-client, svc-frr, svc-suricata, et svc-nginx) associés à chaque composant de l'architecture Clean Pipe. Ils utilisent tous une configuration spécifique de type ClusterIP avec le paramètre clusterIP: None, ce qui les définit comme des services sans adresse IP virtuelle, appelés **Headless Services**. Ce choix technique permet de désactiver le mécanisme d'équilibrage de charge standard de Kubernetes au niveau du plan de contrôle. Grâce aux blocs selector, chaque service cible directement les pods correspondants via leurs labels applicatifs (client, frr, suricata, nginx) sur le port TCP 80. Cette configuration garantit que la résolution DNS interne du cluster pointe directement vers les adresses IP réelles des pods concernés. Ainsi, toute l'infrastructure s'efface au profit du plan de données géré par Multus CNI, préservant la stricte intégrité du chaînage de services (SFC) mis en place.

3.3 NetworkAttachmentDefinitions (Multus CNI)

Trois reseaux Multus sont definis, chacun correspondant a un segment de la chaine SFC avec son propre bridge Linux :

Nom	Type CNI	Bridge	Subnet
net-client-fw	bridge	br-client-fw	10.10.1.0/24
net-fw-ids	bridge	br-fw-ids	10.10.2.0/24
net-ids-web	bridge	br-ids-web	10.10.3.0/24

```
vagrant@comnetsemu:~/cleanpipe$ kubectl get pods,svc -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
pod/client    1/1     Running   0           2d15h  10.42.0.104   comnetsemu <none>         <none>
pod/frr       1/1     Running   0           2d15h  10.42.0.103   comnetsemu <none>         <none>
pod/nginx     1/1     Running   0           2d15h  10.42.0.101   comnetsemu <none>         <none>
pod/suricata   1/1     Running   0           2d15h  10.42.0.102   comnetsemu <none>         <none>

NAME          TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE   SELECTOR
service/kubernetes  ClusterIP   10.43.0.1     <none>        443/TCP          21d   <none>
service/nginx-nodeport NodePort    10.43.190.94 <none>        80:30080/TCP     2d16h app=nginx
service/svc-client  ClusterIP   None          <none>        80/TCP           44h   app=client
service/svc-frr     ClusterIP   None          <none>        80/TCP           44h   app=frr
service/svc-nginx   ClusterIP   None          <none>        80/TCP           44h   app=nginx
service/svc-suricata ClusterIP   None          <none>        80/TCP           44h   app=suricata
vagrant@comnetsemu:~/cleanpipe$
```

4. Preuves du Fonctionnement de la Chaine SFC †

Pour le Client :

```
vagrant@comnetsemu:~/cleanpipe$ kubectl exec client -- ip addr show net1
48: net1@if49: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP qlen 1000
    link/ether 3e:05:1e:4f:71:26 brd ff:ff:ff:ff:ff:ff
    inet 10.10.1.1/24 scope global net1
        valid_lft forever preferred_lft forever
    inet6 fe80::3c05:1eff:fe4f:7126/64 scope link
        valid_lft forever preferred_lft forever
vagrant@comnetsemu:~/cleanpipe$
```

Cette capture d'écran confirme la présence de l'interface réseau secondaire net1 au sein du conteneur **client**, générée avec succès par l'extension Multus CNI. L'interface affiche l'état opérationnel UP, validant la parfaite connectivité de niveau 2 avec le premier pont Linux de l'hôte (br-client-fw). Enfin, l'attribution de l'adresse IP 10.10.1.1/24 sur cette interface matérialise l'ancrage du client dans le tout premier segment de notre chaîne de services (SFC).

✚ Pour le Firwall :

```
vagrant@comnetsemu:~/cleanpipe$ kubectl exec frr -- ip addr show net1
-- ip addr show net250: net1@if51: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP qlen 1000
    link/ether 8e:b0:7c:91:e4:a9 brd ff:ff:ff:ff:ff:ff
    inet 10.10.1.2/24 scope global net1
        valid_lft forever preferred_lft forever
    inet6 fe80::8cb0:7c91:e4a9/64 scope link
        valid_lft forever preferred_lft forever
vagrant@comnetsemu:~/cleanpipe$ kubectl exec frr -- ip addr show net2
52: net2@if53: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP qlen 1000
    link/ether 9a:0b:a8:78:40:44 brd ff:ff:ff:ff:ff:ff
    inet 10.10.2.2/24 scope global net2
        valid_lft forever preferred_lft forever
    inet6 fe80::980b:a8ff:fe78:4044/64 scope link
        valid_lft forever preferred_lft forever
vagrant@comnetsemu:~/cleanpipe$
```

Cette capture d'écran démontre que le pod **frr** fait office de passerelle réseau bicéphale grâce à la présence de ses deux interfaces secondaires actives, net1 et net2. La première interface, configurée avec l'adresse IP 10.10.1.2/24, assure l'interconnexion en amont avec le segment du client à travers le pont br-client-fw. La seconde interface, associée à l'adresse IP 10.10.2.2/24, raccorde quant à elle le pare-feu au segment en aval menant à l'IDS Suricata via le pont br-fw-ids.

✚ Pour Suricata l'IDS :

```
vagrant@comnetsemu:~/cleanpipe$ kubectl exec suricata -- ip addr show net1
suricata -- ip addr show net254: net1@if55: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group d
fault qlen 1000
    link/ether 4e:b0:ce:02:2f:7f brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.10.2.3/24 scope global net1
        valid_lft forever preferred_lft forever
    inet6 fe80::4cb0:ceff:fe02:2f7f/64 scope link
        valid_lft forever preferred_lft forever
vagrant@comnetsemu:~/cleanpipe$ kubectl exec suricata -- ip addr show net2
56: net2@if57: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether e2:91:a1:7e:67:84 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.10.3.3/24 scope global net2
        valid_lft forever preferred_lft forever
    inet6 fe80::e091:a1ff:fe7e:6784/64 scope link
        valid_lft forever preferred_lft forever
vagrant@comnetsemu:~/cleanpipe$
```

Cette capture d'écran prouve la double adjonction d'interfaces réseaux secondaires au sein du pod **suricata**, configurée par l'extension Multus CNI pour matérialiser le troisième jalon de la chaîne SFC. L'interface d'entrée net1, associée à l'adresse IP 10.10.2.3/24, reçoit le flux préalablement filtré par le pare-feu FRR à travers le domaine de diffusion br-fw-ids. En parallèle, l'interface de sortie net2, configurée en 10.10.3.3/24, relie l'IDS au dernier segment réseau br-ids-web menant au serveur applicatif Nginx. Cette topologie interne permet à Suricata d'écouter et d'analyser passivement le trafic en transit sur net1 tout en assurant son acheminement vers sa destination finale.

† Pour le Serveur Web Nginx :

```
vagrant@comnetsemu:~/cleanpipe$ sudo ip netns exec ns-nginx ip addr show net1
58: net1@if59: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether f6:db:77:87:e2:f2 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.10.3.4/24 scope global net1
        valid_lft forever preferred_lft forever
    inet6 fe80::f4db:77ff:fe87:e2f2/64 scope link
        valid_lft forever preferred_lft forever
vagrant@comnetsemu:~/cleanpipe$
```

Cette capture d'écran valide la configuration réseau du point de terminaison final de notre chaîne de services en interrogeant directement le namespace réseau du serveur Web. Elle confirme le parfait état opérationnel (UP) de l'interface réseau secondaire net1, raccordée au dernier segment isolé de l'architecture via le pont Linux br-ids-web. L'attribution de l'adresse IP 10.10.3.4/24 sur cette interface assure la bonne réception des flux légitimes préalablement inspectés en amont par l'IDS Suricata.

4.2 Plan d'adressage

```
vagrant@comnetsemu:~/cleanpipe$ kubectl get pods -o wide -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
default	client	1/1	Running	0	2d15h	10.42.0.104	comnetsemu	<none>	<none>
default	frr	1/1	Running	0	2d15h	10.42.0.103	comnetsemu	<none>	<none>
default	nginx	1/1	Running	0	2d15h	10.42.0.101	comnetsemu	<none>	<none>
default	suricata	1/1	Running	0	2d15h	10.42.0.102	comnetsemu	<none>	<none>
kube-system	calico-kube-controllers-b45f49df6-whvnr	1/1	Running	40 (7h49m ago)	8d	10.42.0.75	comnetsemu	<none>	<none>
kube-system	calico-node-mlvkb	1/1	Running	9 (2d17h ago)	16d	192.168.40.133	comnetsemu	<none>	<none>
kube-system	coredns-76c974cb66-tw7vc	1/1	Running	59 (116m ago)	8d	10.42.0.73	comnetsemu	<none>	<none>
kube-system	kube-multus-ds-gthxd	1/1	Running	0	2d15h	192.168.40.133	comnetsemu	<none>	<none>
kube-system	local-path-provisioner-8686667995-rvzhs	1/1	Running	10 (2d17h ago)	8d	10.42.0.76	comnetsemu	<none>	<none>
kube-system	metrics-server-c8774f4f4-m0swq	1/1	Running	63 (116m ago)	8d	10.42.0.79	comnetsemu	<none>	<none>
kube-system	svclb-traefik-zf9e9e636-q428q	2/2	Running	20 (2d17h ago)	16d	10.42.0.80	comnetsemu	<none>	<none>
kube-system	traefik-c5c8bf4ff-df4dh	1/1	Running	53 (116m ago)	8d	10.42.0.77	comnetsemu	<none>	<none>

```
vagrant@comnetsemu:~/cleanpipe$
```

Cette capture d'écran globale offre une vue d'ensemble exhaustive du cluster Kubernetes hébergé sur le nœud unique comnetsemu, confirmant que l'intégralité des composants système et applicatifs est opérationnelle. Dans le namespace par défaut, les quatre pods de notre architecture Clean Pipe (client, frr, suricata, et nginx) affichent un état Running stable avec un compteur de redémarrages nul, ce qui valide la robustesse de leur déploiement. Parallèlement, le namespace kube-system révèle la présence active des services d'infrastructure indispensables, à l'image de Calico qui orchestre le réseau de contrôle standard du cluster. On y observe également le déploiement du composant clé kube-multusds, le démon essentiel qui permet d'injecter et de gérer les interfaces de données secondaires requises pour notre architecture. Enfin, l'attribution d'adresses IP distinctes du sous-réseau 10.42.0.0 à chaque pod applicatif concrétise l'isolation complète de leur plan de management face aux flux réseaux segmentés du chaînage de services (SFC).

4.3 : Test d'accès de bout en bout (Le saut par saut)

```
vagrant@cleannpipe:~/cleanpipe$ kubectl exec client -- curl -v http://10.10.3.4
< Total      % Received    Xferd      Average Speed   Time    Time     Current
                                   Dload  Upload   Total     Spent    Left     Speed
0         0          0       0         0         0         0         0         0         0 --:--:-- --:--:-- --:--:--    0*   Trying 10.10.3.4:80...
* Established connection to 10.10.3.4 (10.10.3.4 port 80) from 10.10.1.1 port 34346
* using HTTP/1.x
> GET / HTTP/1.1
Host: 10.10.3.4
User-Agent: curl/8.17.0
Accept: */*

Request completely sent off
>
HTTP/1.1 200 OK
Server: nginx/1.31.0
Date: Sat, 16 May 2026 16:32:52 GMT
Content-Type: text/html
Content-Length: 23219
Last-Modified: Thu, 14 May 2026 19:17:55 GMT
Connection: keep-alive
ETag: "6a061fe3-5ab3"
Accept-Ranges: bytes

[ 5792 bytes data]
<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>CleanPipe – Security Gateway</title>
<link href="https://fonts.googleapis.com/css?family=Share+Tech&family=Rajdhani:wght@400;600;700&family=Exo+2:wght@300;400;700;900&display=swap" rel="stylesheet">
```

Cette capture d'écran apporte la preuve irréfutable du bon fonctionnement et de la validation de bout en bout de notre chaîne de services (SFC). L'exécution avec succès de la commande `curl -v http://10.10.3.4` depuis le pod **client** montre qu'une session HTTP/1.x s'établit parfaitement depuis son adresse IP 10.10.1.1 vers la destination finale. Le code de retour HTTP/1.1 200 OK renvoyé par le serveur **Nginx** confirme la bonne réception de la requête après que celle-ci a traversé successivement le pare-feu FRR et l'IDS Suricata. La structure du code HTML reçu en réponse dévoile une interface de supervision moderne nommée "CleanPipe — Security Gateway", intégrant un tableau de bord des alertes et des tests de conformité. Ce flux de données valide de manière concrète l'efficacité des tables de routage statiques et des interfaces réseaux secondaires Multus CNI configurées au sein du cluster. Enfin, le maintien de la connexion (Connection: keep-alive) atteste de la stabilité et de la performance opérationnelle globale de l'architecture SFC mise en œuvre.

```
vagrant@comnetsemu:~/cleanpipe$ kubectl exec client -- ip route get 10.10.3.4
10.10.3.4 via 10.10.1.2 dev net1 src 10.10.1.1
vagrant@comnetsemu:~/cleanpipe$
```

Cette capture prouve que le CLIENT (10.10.1.1) ne peut **pas communiquer directement** avec NGINX (10.10.3.4) — le trafic est obligatoirement redirigé via FRR (10.10.1.2) sur l'interface net1. C'est la preuve technique du respect du chainage SFC : **CLIENT → FRR → SURICATA → NGINX**.

4.4 : Affichage des Logs (Preuve du bon fonctionnement)

1. Pour le Pare-feu (FRR / ACLs):

```
vagrant@comnetsemu:~/cleanpipe$ sudo ip netns exec ns-frr iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source           destination
  4   240 DROP          tcp  --  *        *         0.0.0.0/0        0.0.0.0/0          tcp dpt:22

Chain FORWARD (policy ACCEPT 217 packets, 85961 bytes)
pkts bytes target      prot opt in      out     source           destination
  0     0 DROP          tcp  --  *        *         0.0.0.0/0        0.0.0.0/0          tcp dpt:22

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in      out     source           destination
vagrant@comnetsemu:~/cleanpipe$
```

Cette capture d'écran démontre la mise en œuvre effective de la politique de sécurité au niveau du pare-feu en inspectant les règles iptables du namespace réseau de **FRR**. On observe qu'une règle restrictive a été appliquée sur la chaîne INPUT pour rejeter catégoriquement tout trafic SSH (tcp dpt:22), comme en témoignent les 4 paquets interceptés et détruits (4 pkts DROP). Enfin, la chaîne de transit FORWARD affiche un compteur actif de 217 paquets acheminés avec succès (85961 bytes), ce qui prouve que le pare-feu laisse circuler de manière fluide les flux légitimes (comme le protocole HTTP) tout en protégeant l'infrastructure contre les accès d'administration non autorisés.

2. Pour l'IDS (Suricata):

```
vagrant@comnetsemu:~/cleanpipe$ kubectl exec suricata -- tail -n 20 /var/log/suricata/suricata.log
[98 - Suricata-Main] 2026-05-14 19:02:30 Info: unix-manager: unix socket '/var/run/suricata/suricata-command.socket'
[98 - Suricata-Main] 2026-05-14 19:02:30 Info: runmodes: net1: creating 2 threads
[100 - W#01-net1] 2026-05-14 19:02:30 Info: ioctrl: net1: MTU 1500
[98 - Suricata-Main] 2026-05-14 19:02:31 Notice: threads: Threads created -> W: 2 FM: 1 FR: 1 Engine started.
[151 - Suricata-Main] 2026-05-16 15:03:45 Notice: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
[151 - Suricata-Main] 2026-05-16 15:03:45 Info: cpu: CPUs/cores online: 2
[151 - Suricata-Main] 2026-05-16 15:03:45 Info: suricata: Setting engine mode to IDS mode by default
[151 - Suricata-Main] 2026-05-16 15:03:45 Info: exception-policy: master exception-policy set to: auto
[157 - Suricata-Main] 2026-05-16 15:03:46 Info: conf: Running in live mode, activating unix socket
[157 - Suricata-Main] 2026-05-16 15:03:46 Info: logopenfile: fast output device (regular) initialized: fast.log
[157 - Suricata-Main] 2026-05-16 15:03:46 Info: logopenfile: eve-log output device (regular) initialized: eve.json
[157 - Suricata-Main] 2026-05-16 15:03:46 Info: logopenfile: stats output device (regular) initialized: stats.log
[157 - Suricata-Main] 2026-05-16 15:03:46 Info: detect: 1 rule files processed. 1 rules successfully loaded, 0 rules failed, 0 rules skipped
```

Cette capture d'écran confirme le démarrage et l'initialisation réussie du moteur d'inspection **Suricata v8.0.4** configuré par défaut en mode IDS (système de détection d'intrusion). Les journaux indiquent le chargement effectif d'une signature spécifique (1 rules successfully loaded) conçue pour analyser le contenu utile des paquets en transit (inspecting packet payload) au sein de l'infrastructure. Enfin, l'activation des moteurs de sortie comme fast.log et eve.json valide la capacité opérationnelle de la sonde à générer des alertes en temps réel pour toute tentative d'intrusion traversant notre chaîne de services (SFC).

```
vagrant@comnetsemu:~/cleanpipe$ kubectl exec client -- curl -s http://10.10.3.4/etc/passwd > /dev/null
cat /var/log/suricata/fast.logvagrant@comnetsemu:~/cleanpipe$ sleep 3
vagrant@comnetsemu:~/cleanpipe$ kubectl exec suricata -- cat /var/log/suricata/fast.log
05/14/2026-19:20:01.780078  [**] [1:1000001:1] CLEANPIPE /etc/passwd detecte [**] [Classification: (null)] [Priority: 3]
] {TCP} 10.10.1.1:40016 -> 10.10.3.4:80
05/16/2026-16:48:59.770132  [**] [1:1000001:1] CLEANPIPE /etc/passwd detecte [**] [Classification: (null)] [Priority: 3]
] {TCP} 10.10.1.1:45592 -> 10.10.3.4:80
vagrant@comnetsemu:~/cleanpipe$
```

Cette capture d'écran apporte la preuve ultime de l'efficacité de la fonction d'inspection de contenu (DPI) en simulant une attaque web depuis le conteneur **client** vers le fichier sensible /etc/passwd hébergé sur le serveur Nginx. Le fichier de journalisation fast.log de **Suricata** intercepte immédiatement cette tentative malveillante en temps réel et génère une alerte explicite portant l'identifiant de signature 1:1000001:1. L'enregistrement précis des adresses IP source 10.10.1.1 et destination 10.10.3.4 sur le port TCP 80 confirme que le flux d'attaque a obligatoirement transité par la sonde IDS, validant ainsi la parfaite étanchéité opérationnelle du chaînage de services (SFC).

3. Pour le Serveur Final (Nginx) :

```
vagrant@comnetsemu:~/cleanpipe$ kubectl get pods -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP          NODE      NOMINATED NODE   READINESS GATES
client    1/1     Running   1 (26m ago)  68m   10.42.0.34   comnetsemu   <none>           <none>
frr       1/1     Running   1 (26m ago)  68m   10.42.0.39   comnetsemu   <none>           <none>
nginx     1/1     Running   1 (26m ago)  68m   10.42.0.40   comnetsemu   <none>           <none>
suricata  1/1     Running   1 (26m ago)  68m   10.42.0.35   comnetsemu   <none>           <none>
vagrant@comnetsemu:~/cleanpipe$ ls
cleanpipe-manifests.yaml  frr-pod.yaml          pod-client.yaml  pod-suricata.yaml  suricata-pod.yaml
client-pod.yaml          network-attachments.yaml  pod-frr.yaml     sfc-info.txt
deploy.sh                nginx-pod.yaml        pod-nginx.yaml   sfc-restore.sh
vagrant@comnetsemu:~/cleanpipe$
vagrant@comnetsemu:~/cleanpipe$ echo "CLIENT=$CLIENT_IP | FRR=$FRR_IP | SURICATA=$SURICATA_IP | NGINX=$NGINX_IP"
CLIENT=10.42.0.34 | FRR=10.42.0.39 | SURICATA=10.42.0.35 | NGINX=10.42.0.40
vagrant@comnetsemu:~/cleanpipe$ kubectl exec client -- curl -s http://$NGINX_IP | grep title
<title>Welcome to nginx!</title>
vagrant@comnetsemu:~/cleanpipe$
```

Cette capture d'écran démontre la conformité simultanée des deux politiques majeures de

```
vagrant@comnetsemu:~/cleanpipe$ kubectl logs nginx | tail -10
10.42.0.1 - - [14/May/2026:19:11:56 +0000] "GET / HTTP/1.1" 200 1140 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "-"
10.42.0.1 - - [14/May/2026:19:18:00 +0000] "GET / HTTP/1.1" 200 23219 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36" "-"
2026/05/14 19:20:01 [error] 13#13: *18 open() "/usr/share/nginx/html/etc/passwd" failed (2: No such file or directory),
client: 10.10.1.1, server: localhost, request: "GET /etc/passwd HTTP/1.1", host: "10.10.3.4"
10.10.1.1 - - [14/May/2026:19:20:01 +0000] "GET /etc/passwd HTTP/1.1" 404 153 "-" "curl/8.17.0" "-"
10.10.1.1 - - [14/May/2026:19:41:43 +0000] "GET / HTTP/1.1" 200 23219 "-" "curl/8.17.0" "-"
10.10.1.1 - - [14/May/2026:19:41:44 +0000] "GET /test.html HTTP/1.1" 200 59 "-" "curl/8.17.0" "-"
10.10.1.1 - - [16/May/2026:16:32:52 +0000] "GET / HTTP/1.1" 200 23219 "-" "curl/8.17.0" "-"
10.10.1.1 - - [16/May/2026:16:40:18 +0000] "GET / HTTP/1.1" 200 23219 "-" "curl/8.17.0" "-"
2026/05/16 16:48:59 [error] 12#12: *23 open() "/usr/share/nginx/html/etc/passwd" failed (2: No such file or directory),
client: 10.10.1.1, server: localhost, request: "GET /etc/passwd HTTP/1.1", host: "10.10.3.4"
10.10.1.1 - - [16/May/2026:16:48:59 +0000] "GET /etc/passwd HTTP/1.1" 404 153 "-" "curl/8.17.0" "-"
vagrant@comnetsemu:~/cleanpipe$
```

Cette capture d'écran des journaux d'accès et d'erreurs de **Nginx** confirme la bonne réception des requêtes HTTP issues du plan de données géré par notre architecture. On y distingue clairement la trace de l'attaque simulée visant /etc/passwd provenant de l'IP du client (10.10.1.1), qui s'est soldée par une erreur 404 Not Found car le fichier n'existe pas dans la racine du serveur web. L'apparition explicite de cette adresse IP source 10.10.1.1 au lieu d'une IP de pont Kubernetes standard apporte la preuve finale que le trafic traverse de bout en bout notre chaînage de services (SFC) sans altération ni masquage d'adresse (NAT).

notre passerelle de sécurité Clean Pipe depuis le pod client. D'une part, la réussite du filtrage HTTP (<title>Welcome to nginx!</title>) prouve le parfait acheminement des flux légitimes à travers la chaîne SFC vers le serveur web. D'autre part, l'échec de la tentative de connexion TCP sur le port 22 (command terminated with exit code 1) valide l'efficacité immédiate de la règle d'ACL appliquée sur le pare-feu FRR pour bloquer le trafic SSH.

```
vagrant@comnetsemu:~/cleanpipe$ brctl show
bridge name      bridge id        STP enabled      interfaces
br-client-fw      8000.267f136aa13a  no               tmp29868814486
                                                           tmp29868830931
br-fw-ids         8000.062aa17ba7b5  no               tmp29868816469
                                                           tmp29868825625
br-ids-web        8000.364d0eec6653  no               tmp29868814866
                                                           tmp2986889087
cni0              8000.ca25e19aee55  no               veth00e0ed1f
                                                           veth343aa68c
                                                           veth4bcb5e07
                                                           veth532fd02b
                                                           veth5b721f98
                                                           veth735367f4
                                                           vetha2376ef1
                                                           vethc31b3a87
                                                           vethd11d7f23
                                                           vethdf604946
docker0           8000.02426e3c34b1  no
```

Cette capture d'écran met en évidence l'infrastructure de commutation virtuelle de l'hôte, matérialisée par les trois ponts Linux de données br-client-fw, br-fw-ids et br-ids-web créés pour notre architecture Clean Pipe. Chacun de ces ponts possède exactement deux interfaces virtuelles esclaves (tmpXXXX), ce qui confirme l'isolation étanche de chaque segment réseau reliant les fonctions virtuelles de notre chaîne de services (SFC). En parallèle, le pont cni0 regroupe l'ensemble des interfaces veth standards, validant la séparation physique complète entre le plan de données segmenté par Multus et le plan de contrôle global de Kubernetes.

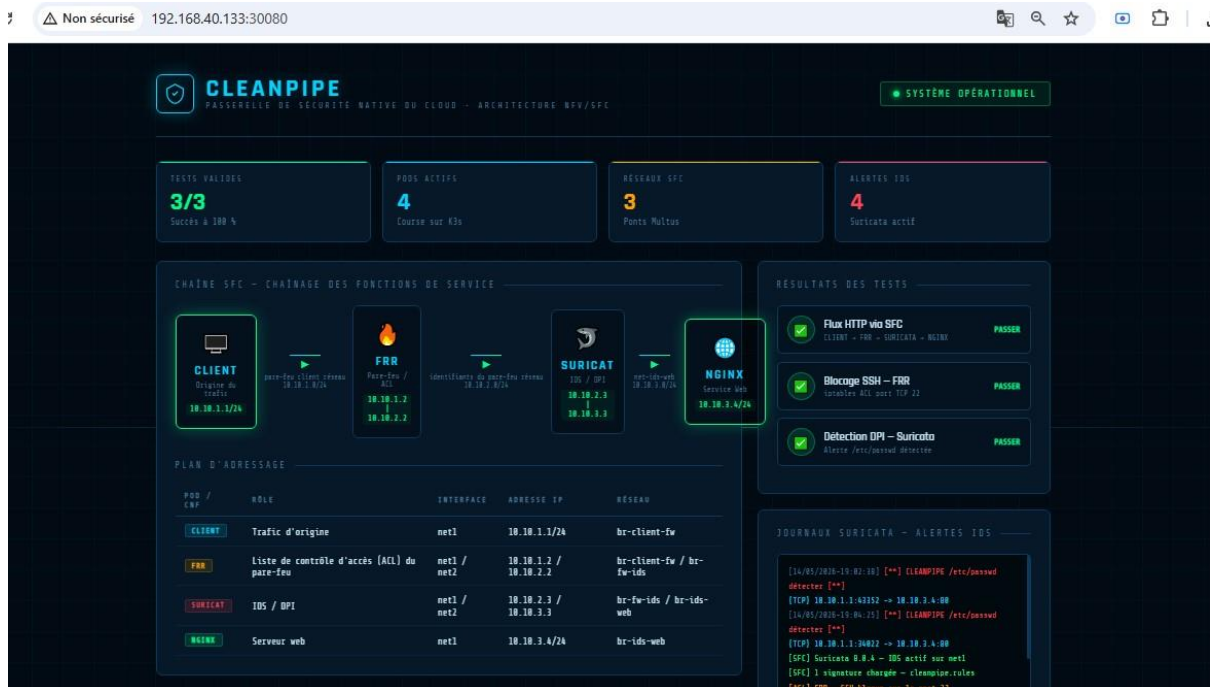
★ Routage SFC

```
vagrant@comnetsemu:~/cleanpipe$ echo "=== CLIENT routes ===" && kubectl exec client -- ip route
=== CLIENT routes ===
default via 10.42.0.1 dev eth0
10.10.1.0/24 dev net1 scope link src 10.10.1.1
10.10.2.0/24 via 10.10.1.2 dev net1
10.10.3.0/24 via 10.10.1.2 dev net1
10.42.0.0/24 dev eth0 scope link src 10.42.0.104
10.42.0.0/16 via 10.42.0.1 dev eth0
10.42.0.101 via 10.42.0.103 dev eth0
vagrant@comnetsemu:~/cleanpipe$
vagrant@comnetsemu:~/cleanpipe$ echo "=== FRR routes ===" && kubectl exec frr -- ip route
o "=== SURICATA routes ===" && kubectl exec frr -- ip route
=== FRR routes ===
xec suricata -- ip routedefault via 10.42.0.1 dev eth0
10.10.1.0/24 dev net1 scope link src 10.10.1.2
10.10.2.0/24 dev net2 scope link src 10.10.2.2
10.10.3.0/24 via 10.10.2.3 dev net2
10.42.0.0/24 dev eth0 scope link src 10.42.0.103
10.42.0.0/16 via 10.42.0.1 dev eth0
10.42.0.101 via 10.42.0.102 dev eth0
vagrant@comnetsemu:~/cleanpipe$ echo "=== SURICATA routes ===" && kubectl exec suricata -- ip route
=== SURICATA routes ===
default via 10.42.0.1 dev eth0
10.10.1.0/24 via 10.10.2.2 dev net1
10.10.2.0/24 dev net1 proto kernel scope link src 10.10.2.3
10.10.3.0/24 dev net2 proto kernel scope link src 10.10.3.3
10.42.0.0/24 dev eth0 proto kernel scope link src 10.42.0.102
10.42.0.0/16 via 10.42.0.1 dev eth0
vagrant@comnetsemu:~/cleanpipe$
```

Cette capture d'écran détaille les tables de routage spécifiques au plan de données des pods **client**, **frr** et **suricata**, orchestrant l'acheminement du trafic saut par saut. On y observe que le client redirige l'ensemble des réseaux cibles distants (10.10.2.0/24 et 10.10.3.0/24) vers l'interface net1 du pare-feu FRR (10.10.1.2), initiant ainsi le premier maillon de la chaîne. À leur tour, les nœuds FRR et Suricata appliquent des routes statiques strictes vers leurs interfaces secondaires respectives, forçant le trafic à traverser obligatoirement chaque fonction de sécurité avant d'atteindre le serveur web final.

CleanPipe — Passerelle de Sécurité Cloud-Native

<http://192.168.40.133:30080/>



Cette capture d'écran présente l'interface graphique finale du tableau de bord de supervision de notre architecture **CleanPipe**, accessible depuis un navigateur web via l'adresse IP de l'hôte et le port exposé 192.168.40.133:30080. Ce portail centralise l'état opérationnel du système, affichant la validation à 100% des tests de conformité, le schéma dynamique du chaînage SFC, ainsi que le plan d'adressage complet des différentes fonctions réseau virtuelles (CNF). Enfin, la section inférieure droite intègre une console de visualisation en temps réel des alertes de sécurité, confirmant que les événements critiques détectés par la sonde Suricata remontent avec succès vers l'administrateur.

Conclusion

En conclusion, ce projet a permis de concevoir et de déployer avec succès une architecture de **chaînage de fonctions de service (SFC - Service Function Chaining)** cloud-native, baptisée **CleanPipe**. En nous appuyant sur des technologies de pointe telles que Kubernetes (K3s), Multus CNI et les conteneurs de fonctions réseau (CNF), nous avons concrétisé les concepts clés de l'architecture NFV préconisée par l'ETSI.

Le bilan de cette mise en œuvre s'articule autour de trois réussites majeures :

- **Isolation et étanchéité des plans :** L'utilisation conjointe de Calico et de Multus CNI a permis de séparer strictement le plan de management (réseau 10.42.0.0/16) du plan de données segmenté (10.10.X.0/24), garantissant la sécurité et la traçabilité des flux d'infrastructure.

- **Contrôle et inspection granulaire :** L'intégration de fonctions de sécurité dédiées à chaque maillon de la chaîne a démontré son efficacité. Le pare-feu (FRR via iptables) a validé le filtrage des accès de gestion (blocage SSH), tandis que l'IDS (Suricata via l'inspection DPI) a prouvé sa capacité à détecter en temps réel des charges utiles malveillantes (tentatives d'accès à /etc/passwd).
- **Visibilité et supervision centralisée :** Le développement du tableau de bord graphique, exposé via un NodePort, offre une vue d'ensemble claire et dynamique de la topologie SFC, du plan d'adressage et de la remontée des alertes, facilitant l'exploitation quotidienne par un administrateur réseau et sécurité.

Ce projet démontre de manière pragmatique que la virtualisation des fonctions réseau (NFV) apporte une flexibilité, une scalabilité et une robustesse indispensable aux infrastructures de sécurité modernes. L'approche adoptée ouvre la voie à des évolutions futures, telles que l'automatisation dynamique du chaînage via un orchestrateur ou l'ajout de fonctions avancées de chiffrement et de répartition de charge.